PARTE I. Practicando con excepciones

1. En su directorio descarguen los archivos contenidos en neoFlix.zip, revisen el contenido y estudien el diseño estructural de la aplicación (únicamente la zona en azul).

- Content (Clase Base): Actúa como la superclase abstracta del modelo, definiendo los atributos generales y los contratos de métodos para el cálculo de estadísticas.
- Audiovisua:: Clase abstracta derivada de Content que incorpora métricas de consumo e interacción del usuaroi. Sus subclases concretas son:
    - Movie: Agrega el atributo de presupuesto (budge).
    - Episode: Representa capítulos individuales de una serie y mantiene una asociación bidireccional con la clase Series.
- Series: Subclase de Content que gestiona una colección ordenada (ordered) de objetos de tipo Episode
- Collection: Permite agrupar elementos bajo un tema específico (theme). Al heredar de Content y mantener a su vez una relación con la clase base

2. Expliquen por qué el proyecto no compila. Realicen las adiciones necesarias para lograrlo.
   Porque tocaba hacer la clase excepción parar lograr que compile

```java
package domain;

/**
 * Excepción propia del programa NeoFlix.
 * Se usa cuando algo sale mal.
 *
 * @author Juan David Rojas Heredia
 */
public class NeoFlixException extends Exception {

    /** Mensaje para cuando un valor no se encuentra. */
    public static final String VALUE_UNKNOWN = "Valor no encontrado";

    /** Mensaje para cuando hay errores en los datos. */
    public static final String DATA_ERROR = "Hay errores en los datos";

    /** Mensaje para cuando algo todavía no se ha implementado. */
    public static final String TO_BE_IMPLEMENTED = "Debe ser implementado";

    /** Mensaje para cuando el contenido está vacío. */
    public static final String CONTENT_EMPTY = "El contenido esta vacio";

    /**
     * Crea una excepción con el mensaje que le pasemos.
     *
     * @param meesage el mensaje que explica el error
     */
    public NeoFlixException(String meesage)
    {
        super(meesage);
    }
}
```

3. Dado el diseño y las pruebas, documenten (si es necesario) y codifiquen el método raiting().
4. Dada la documentación y el diseño, codifiquen y prueben el método raiting(default).
5. Documenten, diseñen, codifiquen y prueben el método raiting(previous).
6. Propongan el cálculo, documenten, diseñen, codifiquen y prueben el método popularity(). (Definan reglas que obliguen a utilizar excepciones en sus cuatro momentos: propagar, lanzar y atender)

PARTE II. Desarrollando

A. Revisando línea base

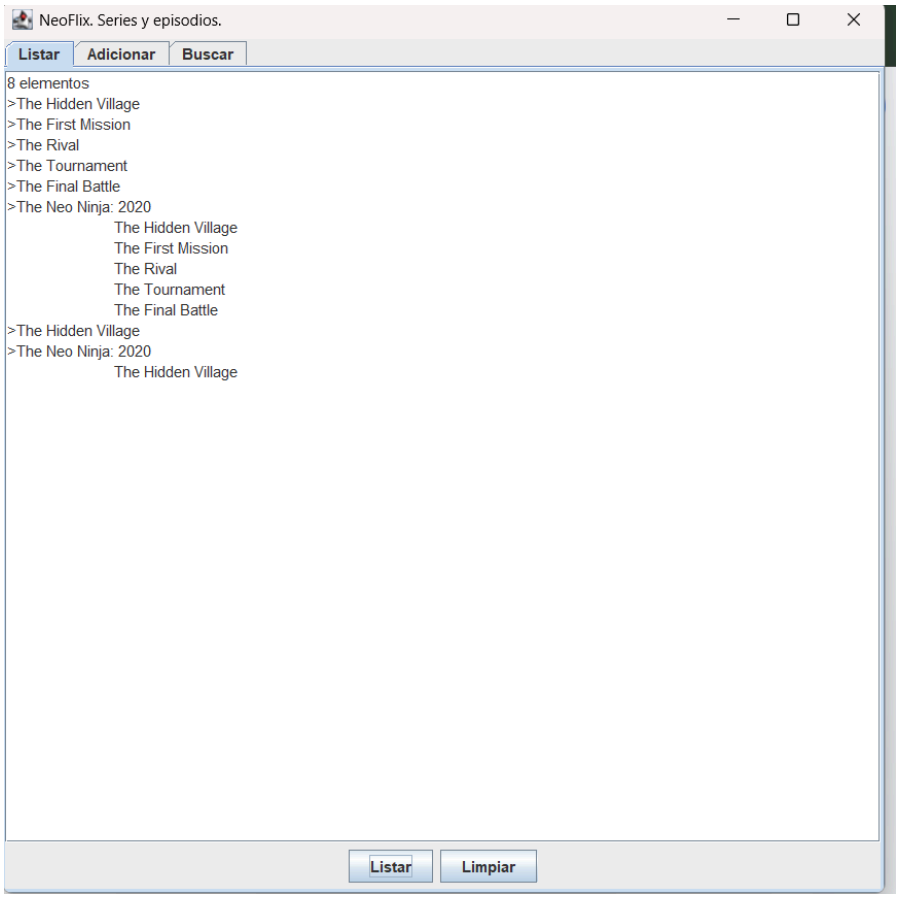
1. En su directorio descarguen los archivos contenidos en neoFlixMore.zip, revisen el contenido. (a) ¿Cuántos archivos se tienen? (b) ¿Cómo están organizados? (c) ¿Cómo deberían estar organizados?
    a. Tiene 3 archivos
    b. Sueltos en una sola carpeta, sin subcarpetas.
    c. Mirando la primera linea de cada uno dice que deberian estar divididos en paquetes. Log y NeoFlix declaran package domain y NeoFlixGUI declara package presentation
2. Estudien el diseño del programa: diagramas de paquetes y de clases. (a) ¿Cuántos paquetes tenemos? (b) ¿Cuántas clases tiene el sistema? (c) ¿Cómo están organizadas? (d) ¿Cuál es la clase ejecutiva?
    a. El programa tiene 2 paquetes domain, presentation. Sin embargo tambien tiene el paquete de test
    b. El sistema tendria 10 clases. domain tendría 9 (Content, Audiovisual, Episode, Movie, Series, Collection, NeoFlixException, NeoFlix, Log) y presentation tendría 1 (NeoFlixGUI)
    c. Estan divididas en paquetes
    d. La clase ejecutiva es NeoFlixGUI
3. Incluya los nuevos archivos en los directorios correspondientes para ejecutar el proyecto, (a) ¿Qué clases deben tener cada directorio? (c) ¿Cómo los incluyeron?
    a. El directorio domain tiene (Content, Audiovisual, Episode, Movie, Series, Collection, NeoFlixException, NeoFlix, Log) y presentation tiene (NeoFlixGUI)
    b. Para agregar las clases a domain unicamente es acceder al paquete y dar click derecho y escoger la opción “Agregar Clase desde Archivo…” buscar donde estaba la carpeta neoFlixMore y escoger las dos clases extras de este paquete. Para NeoFlixGUI primero se creo el paquete de presentation y luego se hizo el mismo proceso.
4. Ejecute el proyecto, (a) ¿Qué funcionalidades ofrece? (b) ¿Cuáles funcionan?
    a. Ofrece Listar, Adicionar y Buscar
    b. Listar y Adicionar
5. Revisen el código y la documentación del proyecto. (a) ¿De dónde salen los datos iniciales? (b) ¿Qué clase pide que se adicionen? (c) ¿Qué clase los adiciona?
    a. Los datos iniciales salen de NeoFlix
    b. La clase que pide que se adicionen es NeoFlixGUI
    c. La clase que lo adicionan NeoFlix en su constructor

B. Realizando inversa. Adicionar y listar. Todo OK.

El objetivo es realizar ingeniería inversa a las funciones de adicionar y listar.

1. Adicionen un nuevo episodio y un nueva serie.
    Serie: The Neo Ninja
    Episodio The Hidden Village
    (a) ¿Qué ocurre? (b) ¿Cómo lo comprueban? Capturen la pantalla. (c) ¿Es adecuado este comportamiento?

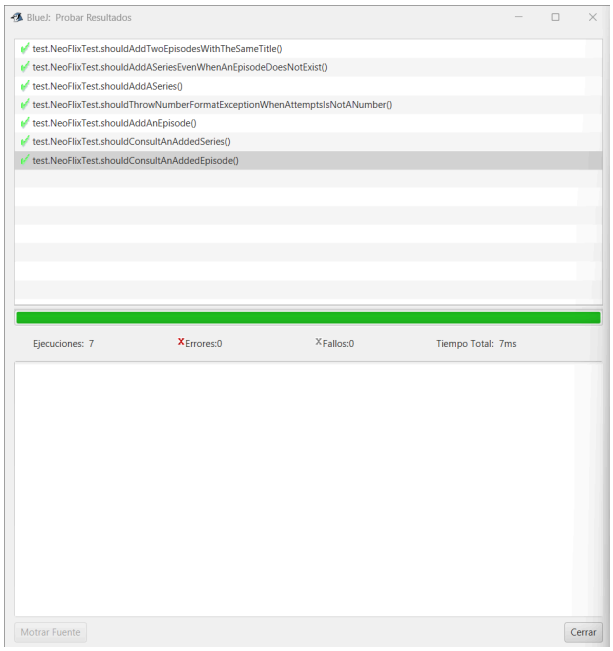
a. Se crean contenidos con el mismo nombre que otros que ya existían, y el programa no avisa.
b. Se comprueba escogiendo la pestaña de Listar



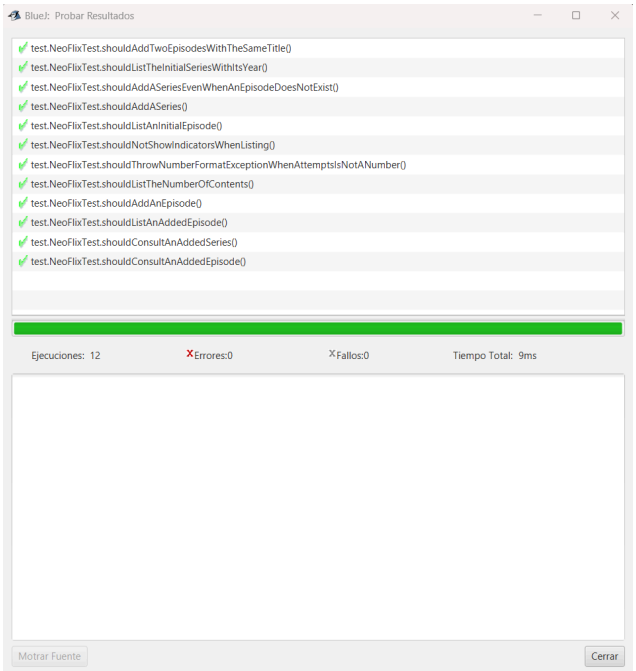
c. No es adecuado: el programa permitió repetir nombres sin avisar, y eso deja una de las dos series inalcanzable al consultar y deja episodios repetidos con datos distintos. Debería rechazar el nombre repetido con un mensaje al usuario.

2. Revisen el código asociado a adicionar en la capa de presentación y la capa de dominio. (a) ¿Qué método es responsable en la capa de presentación? (b) ¿Qué método en la capa de dominio?
   a. El metodo responsable es actionAdd()
   b. En dominio llamo a addEpisode y addSeries

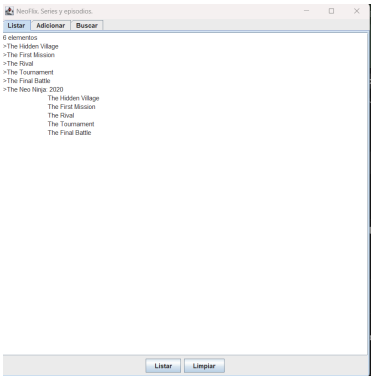
3. Realicen ingeniería inversa para la capa de dominio para adicionar. Capturen los resultados de las pruebas de unidad.

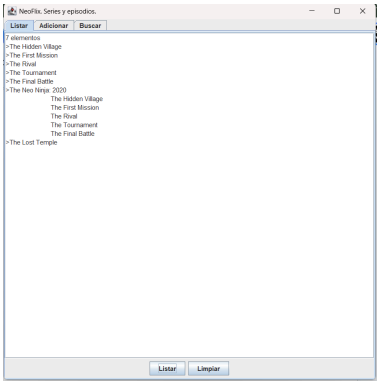


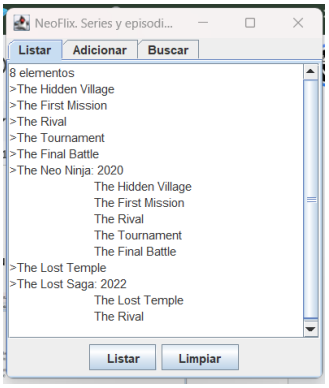
4. Revisen el código asociado a listar en la capa de presentación y la capa de dominio. (a) ¿Qué método es responsable en la capa de presentación? (b) ¿Qué método en la capa de dominio?
    a. El método que es responsable en presentacion es actionList()
    b. El método que es responsable en la capa de dominio es toString()
5. Realicen ingeniería reversa para la capa de dominio para listar. Capturen los resultados de las pruebas de unidad.



6. Propongan y ejecuten una prueba de aceptación. Capturen las pantallas correspondientes.







1. El programa inicializa con 6 elementos, con los 5 episodios y The Neo Ninja: 2020
2. Se adiciona un episodio válido Nombre The Lost Temple, Intentos 100, Completa 60, Votos 50, Suma 354.
3. En listar salen 7 elementos, y >The Lost Temple al final
4. Se adiciona una serie con el episodio creado y uno ya existe. Nombre The Lost Saga, Año 2022, Episodios (uno por línea) The Lost Temple y The Rival.
5. En listar el resultado se deben mostrar 8 elementos, incluyendo el episodio nuevo y la serie con sus dos episodios.

C. Fortaleciendo. Adicionar robusta

1. Propongan una prueba de aceptación que genere el fallo. (a) ¿Se generó alguna excepción? (b) ¿Por qué?
   Prueba de aceptacion:
   (i) Nombre repetido
   Nombre: The Hidden Village, intentos 120, completos 50, votos 430, suma 1000
   (ii) No numérico
   Nombre: Nuevo, intentos abc, completos 50, votos 430, suma 1000
   (iii) Valores incoherentes
   Nombre: Malo, intentos 10, completos 20, votos 2, suma 50
   (iv): serie con un episodio que no existe
   Nombre: The Ninja Returns Año: 2021 Episodios The Hidden Village Episodio Fantasma

   1.1. ¿Y si el nombre ya existe?
      a. No. Se adiciona sin ningún aviso. En Listar aparecerá “The Hidden Village” dos veces.
      b. Porque addEpisode en NeoFlix nunca revisa si el título ya existe. Hace contents.add(e), y episodes.put(title.toUpperCase(), e) reemplaza en silencio el episodio anterior en el TreeMap. Por eso el TreeMap y el ArrayList quedan inconsistentes.

   1.2. ¿Y si los valores numéricos no son numéricos?
      a. Sí, pero la lanza Java. NumberFormatException.

b. addEpisode recibe todo como String y convierte con parseInt sin try/catch. NumberFormatException es una excepción no verificada (hereda de RuntimeException). Java no obliga a atenderla, y ni addEpisode ni actionAdd en NeoFlixGUI lo hacen

1.3. ¿Y si los valores no son los esperados? (suma de votos>10*votos o terminados>intentos)
   a. No. Se adiciona sin ningún aviso.
   b. Porque nunca se verifican las condiciones 0<=completed<=attempts, sumVotes<=votes*10 al adicionar los chequeos solo exiten en popularity() y rating()

1.4. Serie con un episodio que no existe
   a. Al adicionar no, y la pantalla no muestra nada. La excepción aparece después, al listar: NullPointerException.
   b. En NeoFlix.addSeries, episodes.get("EPISODIO FANTASMA") devuelve null, porque ese título no está en el TreeMap. Nadie lo revisa. Series.addEpisode(null) tampoco valida, así que guarda el null en la lista de episodios. contents.add(s) se ejecuta igual, y la serie queda guardada con un null dentro. Al listar, Series.data recorre sus episodios y llama e.data(...) sobre ese null, lo que lanza el NullPointerException.

2. Analicen el diseño dinámico (diagrama de secuencia). Para hacer la funcionalidad robusta: (a) ¿Qué método debería lanzar la excepción? (b) ¿Qué métodos deberían propagarla? (c) ¿Qué método debería atenderla? Expliquen claramente.

2.1. ¿Y si el nombre ya existe?
   a. Lanza: NeoFlix.addEpisode
   b. Propaga: ningún método.
   c. Atiende: NeoFlixGUI.actionAdd

2.2. ¿Y si los valores numéricos no son numéricos?
   a. Lanza: NeoFlix.addEpisode
   b. Propaga: ningún método
   c. Atiende: NeoFlixGUI.actionAdd

2.3. ¿Y si los valores no son los esperados? (suma de votos>10*votos o terminados>intentos)
   a. Lanza: el constructor de Audiovisual (invocado desde el constructor de Episode), con NeoFlixException(DATA_ERROR)
   b. Propaga: el constructor de Episode y NeoFlix.addEpisode
   c. Atiende: NeoFlixGUI.actionAdd

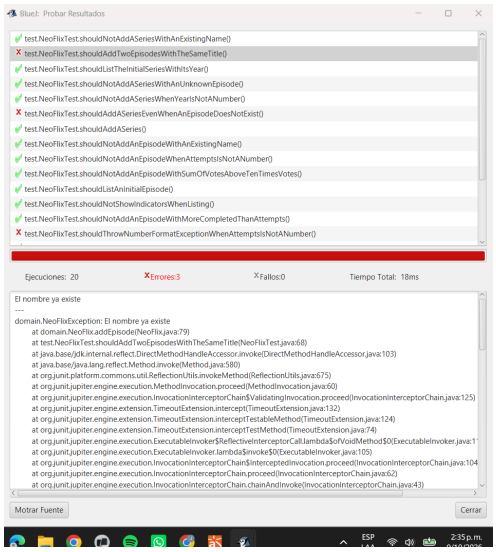
2.4. Serie con un episodio que no existe
   a. Lanza: NeoFlix.addSeries, con NeoFlixException(EPISODE_NOT_FOUND). Es quien consulta el TreeMap de episodios y recibe null cuando el título no existe
   b. Propaga: ningún método, porque NeoFlixGUI.actionAdd llama directamente a NeoFlix.addSeries.
   c. Atiende: NeoFlixGUI.actionAdd

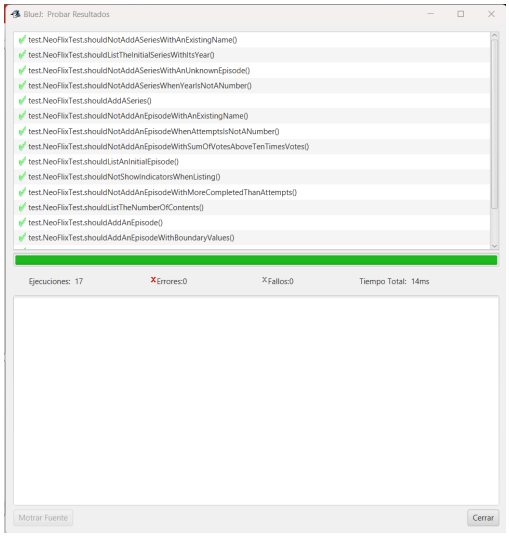
3. Construya la solución propuesta. Incluyan las excepciones en el diseño y presente evidencia de las pruebas de unidad: una ejecución normal y una para cada excepción definida.

```java
/** Mensaje para cuando el nombre ya existe. */
public static final String NAME_EXISTS = "El nombre ya existe";

/** Mensaje para cuando un valor no es numérico. */
public static final String NUMBER_FORMAT = "Los valores deben ser numericos";

/** Mensaje para cuando un episodio de la serie no existe. */
public static final String EPISODE_NOT_FOUND = "El episodio no existe";
```






```java
// (iii) Valores incoherentes
@Test
public void shouldNotAddAnEpisodeWithMoreCompletedThanAttempts() {
    try {
        n.addEpisode("Malo", "?", "10", "20", "2", "15");
        fail("Did not throw exception");
    } catch (NeoFlixException e) {
        assertEquals(NeoFlixException.DATA_ERROR, e.getMessage());
    }
    assertEquals(6, n.numberContents());
}

@Test
public void shouldNotAddAnEpisodeWithSumOfVotesAboveTenTimesVotes() {
    try {
        n.addEpisode("Malo", "?", "10", "5", "2", "50");
        fail("Did not throw exception");
    } catch (NeoFlixException e) {
        assertEquals(NeoFlixException.DATA_ERROR, e.getMessage());
    }
    assertEquals(6, n.numberContents());
}

// (iv) Episodio inexistente
@Test
public void shouldNotAddASeriesWithAnUnknownEpisode() {
    try {
        n.addSeries("The Ninja Returns", "2021", "The Hidden Village\nEpisodio Fantasma");
        fail("Did not throw exception");
    } catch (NeoFlixException e) {
        assertEquals(NeoFlixException.EPISODE_NOT_FOUND, e.getMessage());
    }
    assertEquals(6, n.numberContents());
    assertNull(n.consult("The Ninja Returns"));
}
```

```java
// Adicionar robusta
@Test
public void shouldAddAnEpisodeWithBoundaryValues() throws NeoFlixException {
    n.addEpisode("limite", "?", "10", "10", "5", "50");   // terminados=intentos, suma=10*votos
    assertEquals(7, n.numberContents());
}

// (i) Nombre repetido
@Test
public void shouldNotAddAnEpisodeWithAnExistingName() {
    try {
        n.addEpisode("The Hidden Village", "?", "100", "60", "50", "354");
        fail("Did not throw exception");
    } catch (NeoFlixException e) {
        assertEquals(NeoFlixException.NAME_EXISTS, e.getMessage());
    }
    assertEquals(6, n.numberContents());
}

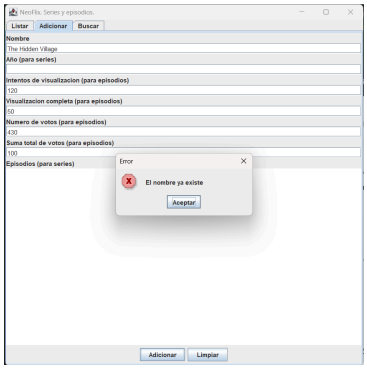
@Test
public void shouldNotAddASeriesWithAnExistingName() {
    try {
        n.addSeries("the neo ninja", "2021", "The Rival");
        fail("Did not throw exception");
    } catch (NeoFlixException e) {
        assertEquals(NeoFlixException.NAME_EXISTS, e.getMessage());
    }
    assertEquals(6, n.numberContents());
}

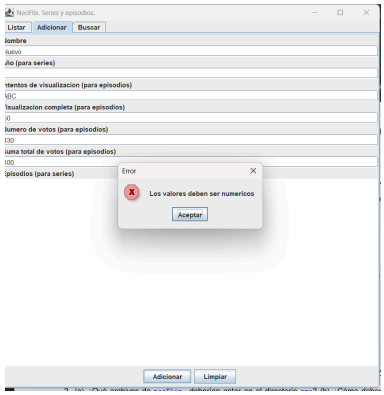
// (ii) Valores no numéricos
@Test
public void shouldNotAddAnEpisodeWhenAttemptsIsNotANumber() {
    try {
        n.addEpisode("Nuevo", "?", "abc", "60", "50", "354");
        fail("Did not throw exception");
    } catch (NeoFlixException e) {
        assertEquals(NeoFlixException.NUMBER_FORMAT, e.getMessage());
    }
    assertEquals(6, n.numberContents());
}

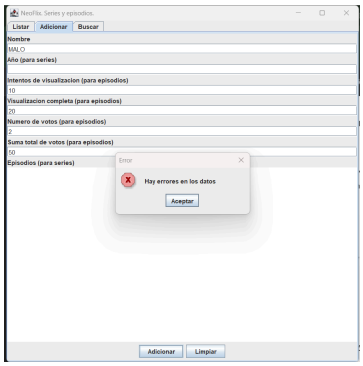
@Test
public void shouldNotAddASeriesWhenYearIsNotANumber() {
    try {
        n.addSeries("Serie Nueva", "dos mil", "The Rival");
        fail("Did not throw exception");
    } catch (NeoFlixException e) {
        assertEquals(NeoFlixException.NUMBER_FORMAT, e.getMessage());
    }
    assertEquals(6, n.numberContents());
}
```

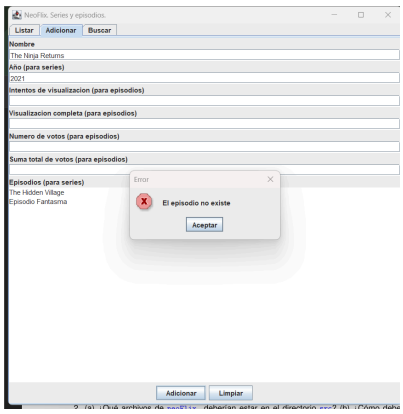
4. Ejecuten nuevamente la aplicación con el caso de aceptación propuesto en 1. (a) ¿Qué sucede ahora? Capturen la pantalla.

    a. Ahora lo que esta sucediendo es que no le esta permitiendo al usuario poder agregar esos casos erróneos mostrándole un aviso



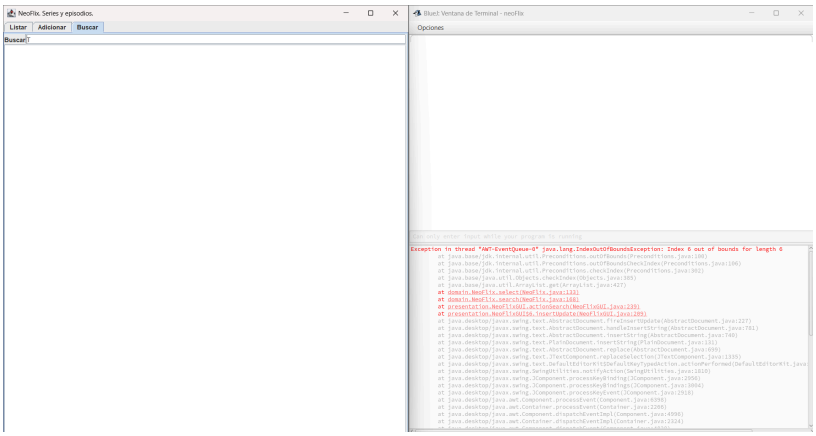




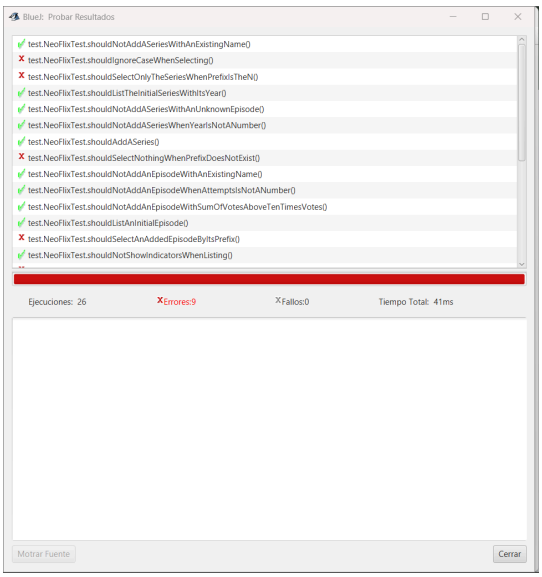


D Registrando errores. Consultar por patrones. ¡Error!

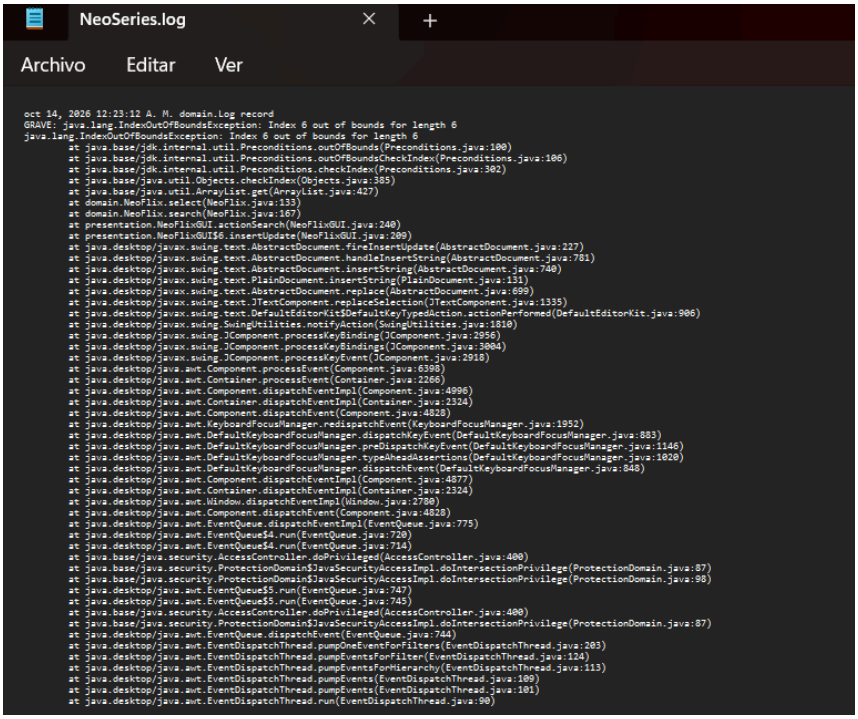
1. Consulten un contenido que inicia por T.(a) ¿Qué sucede? (b) ¿Qué error existe? Capturen el resultado. (d) ¿Quién debe conocer y quien NO debe conocer esta información?
    a. Escribo “T” en Buscar y no aparece nada. La pantalla se queda vacía o con el texto de la búsqueda. No hay ningún mensaje para el usuario y en la consola de BlueJ aparece un error.
    b. Con 6 contenidos, el índice i llega hasta 5, y en esa última vuelta se llama contents.get(6). Eso lanza IndexOutOfBoundsExceptio en NeoFlix.select



    c. respuesta:
        - Debe conocerla el equipo de desarrollo conociendo el tipo de excepción, traza, qué método falló y cuándo.
        - No debe conocerla el usuario final con ese detalle técnico ya que a él le sirve un aviso claro y corto.
2. Revisen el código asociado a consultar en la capa de presentación y la capa de dominio. (a) ¿Qué método es responsable en la capa de presentación? (b) ¿Qué método en la capa de dominio?
    a. actionSearch()
    b. search(String prefix)
3. Realicen ingeniería reversa para la capa de dominio para consultar. No modifiquen el código. Capturen los resultados de las pruebas de unidad.



4. Exploren el método record de la clase Log (a) ¿Qué servicio presta?
   Log.record presta el servicio de registrar en un archivo de texto (NeoSeries.log) los errores inesperados, con fecha, hora, nivel y traza completa, para que el equipo de desarrollo pueda revisarlos después sin mostrarle esos detalles al usuario
5. Analicen el punto adecuado para que EN ESTE CASO se presente un mensaje especial de alerta al usuario, se guarde la información del error en el registro y continúe la ejecución. Expliquen y construyan la solución.
   En actionSearch(). Ya que los métodos de dominio no deben mostrar ventanas. Ademas Es el último método antes de que el error llegue a Swing y es el primero que conoce la pantalla.
6. Ejecuten nuevamente la aplicación con el caso propuesto en 1. (a) ¿Qué mensaje salió en pantalla? (b) ¿La aplicación termina? (c) ¿Qué información tiene el archivo de errores?
   a. El mensaje que aparece fue el que defini que saliera “Ocurrió un problema inesperado en la búsqueda.“
   b. La aplicación sigue abierta
   c. Fecha y hora, nivel grave, el tipo de excepción con su mensaje, y la traza completa que ubica el fallo en NeoFlix.select.



7. Revisen el código, (a) ¿Qué otro caso puede generar un error? (b) ¿Cómo lo solucionarían?
   a. Si se crea un episodio con el nombre vacío y luego otro episodio, `addEpisode` lanza `ClassCastException`. La línea `(Series) consult(series)` busca un contenido con título vacío y encuentra el episodio sin nombre, que no es una serie.

b. **Se propusieron dos soluciones para el caso especifico y el log para un caso mas generar.**
    - En los métodos addEpisode y addSeries de NeoFlix se agrega una validación al inicio: si el título está vacío, se lanza una NeoFlixException con el mensaje DATA_ERROR. Así nunca se guarda un contenido sin nombre, y consult("") deja de encontrar un episodio donde addEpisode espera una serie. Con esto se evita el ClassCastException de este caso. Como es una excepción esperada de negocio, actionAdd ya la atrapa y le muestra el mensaje al usuario.
    - Atender cualquier error inesperado en la capa de presentación. En actionAdd se agrega un segundo catch (Exception e), después del de NeoFlixException. Ahí se registra el error con Log.record(e) y se muestra un mensaje general al usuario, sin detalles técnicos. Así, si ocurre otro error que no previmos, el equipo de desarrollo queda informado por el archivo de registro, el usuario recibe un aviso en lugar de ver la pantalla sin respuesta, y la aplicación continúa.

8. (a) ¿Es adecuado que la aplicación continúe su ejecución después de sufrir un incidente como este? (b) ¿De qué dependería continuar o parar?
    a. En este caso sí. El error ocurrió en una consulta, que solo lee datos y no los modifica, por lo que el estado de la aplicación no quedó dañado. Además, el error se registró en el archivo NeoSeries.log, el usuario fue avisado con un mensaje claro y las demás funciones siguen disponibles. Terminar toda la aplicación por una búsqueda fallida sería una reacción desproporcionada.
    b. Dependería de varios factores:
        - si el estado de los datos pudo quedar inconsistente, como cuando el error ocurre a mitad de una operación que modifica información, caso en el que conviene parar o deshacer los cambios
        - Si el error se repite constantemente, porque entonces seguir degrada la experiencia del usuario y puede ser mejor desactivar esa función
        - qué tan crítica es la aplicación y qué costo tiene un dato erróneo
        - Si el error se pudo registrar, ya que sin registro se pierde la información para corregirlo
        - Si el error es del programa (afecta una acción) o del entorno (falta de memoria, disco lleno) en cuyo caso continuar puede no ser posible.
9. Modifiquen la aplicación para garantizar que SIEMPRE que haya un error no esperado en presentación se maneje de la forma adecuada. (a) ¿Cuál fue la solución seleccionada?

    Se centralizó el manejo de los errores inesperados en la capa de presentación, en un único método privado de NeoFlixGUI llamado handleUnexpected. Este método registra el error en el archivo NeoSeries.log con Log.record(e) y le muestra al usuario un mensaje general, sin detalles técnicos. Así el equipo de desarrollo conserva la información y el usuario no ve la traza. Este método lo usan todas las acciones de la interfaz (actionList, actionAdd y actionSearch). En actionAdd se agregó un segundo

catch (Exception e) después del de NeoFlixException, para que los errores esperados de negocio sigan mostrando su propio mensaje y solo los inesperados pasen por handleUnexpected. Como respaldo, en el main se registró un manejador global de excepciones no atrapadas (Thread.setDefaultUncaughtExceptionHandler), que también llama a handleUnexpected. Con esto, si ocurre un error en cualquier otro punto de la interfaz que no tenga su propio catch, igualmente se registra en el log y se avisa al usuario, y la aplicación continúa en ejecución.

PARTE III. DE BLUEJ A CONSOLA

A Comandos básicos del sistema operativo [En lab04.pdf]

Antes de iniciar debemos repasar los comandos básicos del manejo de la consola.

1. Investiguen los comandos para moverse en la estructura de directorios: crear, borrar, listar su contenido y copiar o eliminar un archivo.

Qué hace	Windows (CMD)	Linux / Mac
Ver en qué carpeta se esta	cd	pwd
Listar contenido	dir	ls
Entrar a una carpeta	cd nombre	cd nombre
Subir un nivel	cd ..	cd ..
Crear carpeta	mkdir nombre	mkdir nombre
Borrar carpeta	rmdir nombre	rmdir nombre
Copiar archivo	copy origen destino	cp origen destino
Eliminar archivo	del archivo	rm archivo

2. Organicen un nuevo directorio con la estructura propuesta para probar desde allí los comandos de consola. Consulten y capturen el contenido de su directorio.

neoFlix

src

domain

presentation

test

```
Lenovo@DESKTOP-PGQD70S MINGW64 ~/Desktop/Universidad Escuela de ingenieros/4. Cuarto Semestre/DOPO/Labo
ratorio-DOPO/RojasH-Lab04/Parte tres (feature)
$ mkdir neoFlix

Lenovo@DESKTOP-PGQD70S MINGW64 ~/Desktop/Universidad Escuela de ingenieros/4. Cuarto Semestre/DOPO/Labo
ratorio-DOPO/RojasH-Lab04/Parte tres (feature)
$ ls
neoFlix/

Lenovo@DESKTOP-PGQD70S MINGW64 ~/Desktop/Universidad Escuela de ingenieros/4. Cuarto Semestre/DOPO/Labo
ratorio-DOPO/RojasH-Lab04/Parte tres (feature)
$ cd neoFlix/

Lenovo@DESKTOP-PGQD70S MINGW64 ~/Desktop/Universidad Escuela de ingenieros/4. Cuarto Semestre/DOPO/Labo
ratorio-DOPO/RojasH-Lab04/Parte tres/neoFlix (feature)
$ mkdir src

Lenovo@DESKTOP-PGQD70S MINGW64 ~/Desktop/Universidad Escuela de ingenieros/4. Cuarto Semestre/DOPO/Labo
ratorio-DOPO/RojasH-Lab04/Parte tres/neoFlix (feature)
$ ls
src/

Lenovo@DESKTOP-PGQD70S MINGW64 ~/Desktop/Universidad Escuela de ingenieros/4. Cuarto Semestre/DOPO/Labo
ratorio-DOPO/RojasH-Lab04/Parte tres/neoFlix (feature)
$ cd src/

Lenovo@DESKTOP-PGQD70S MINGW64 ~/Desktop/Universidad Escuela de ingenieros/4. Cuarto Semestre/DOPO/Labo
ratorio-DOPO/RojasH-Lab04/Parte tres/neoFlix/src (feature)
$ mkdir domain

Lenovo@DESKTOP-PGQD70S MINGW64 ~/Desktop/Universidad Escuela de ingenieros/4. Cuarto Semestre/DOPO/Labo
ratorio-DOPO/RojasH-Lab04/Parte tres/neoFlix/src (feature)
$ mkdir presentation

Lenovo@DESKTOP-PGQD70S MINGW64 ~/Desktop/Universidad Escuela de ingenieros/4. Cuarto Semestre/DOPO/Labo
ratorio-DOPO/RojasH-Lab04/Parte tres/neoFlix/src (feature)
$ mkdir test

Lenovo@DESKTOP-PGQD70S MINGW64 ~/Desktop/Universidad Escuela de ingenieros/4. Cuarto Semestre/DOPO/Labo
ratorio-DOPO/RojasH-Lab04/Parte tres/neoFlix/src (feature)
$ ls
domain/  presentation/  test/

Lenovo@DESKTOP-PGQD70S MINGW64 ~/Desktop/Universidad Escuela de ingenieros/4. Cuarto Semestre/DOPO/Labo
ratorio-DOPO/RojasH-Lab04/Parte tres/neoFlix/src (feature)
$
```

3. En el directorio src/domain copien únicamente el archivo Serie.java del paquete de dominio. Consulten y capturen el contenido de src/domain

```
Lenovo@DESKTOP-PGQD70S MINGW64 ~/Desktop/Universidad Escuela de ingenieros/4. Cuarto Semestre/DOPO/Labo
ratorio-DOPO/RojasH-Lab04/Parte tres/neoFlix/src/domain (feature)
$ cp '/c/Users/Lenovo/Desktop/Universidad Escuela de ingenieros/4. Cuarto Semestre/DOPO/Laboratorio-DOP
O/RojasH-Lab04/neoFlix/neoFlix/domain/Series.java' .

Lenovo@DESKTOP-PGQD70S MINGW64 ~/Desktop/Universidad Escuela de ingenieros/4. Cuarto Semestre/DOPO/Labo
ratorio-DOPO/RojasH-Lab04/Parte tres/neoFlix/src/domain (feature)
$ ls
Series.java

Lenovo@DESKTOP-PGQD70S MINGW64 ~/Desktop/Universidad Escuela de ingenieros/4. Cuarto Semestre/DOPO/Labo
ratorio-DOPO/RojasH-Lab04/Parte tres/neoFlix/src/domain (feature)
$
```

B Estructura de proyectos java

En java los proyectos se estructuran considerando tres directorios básicos.

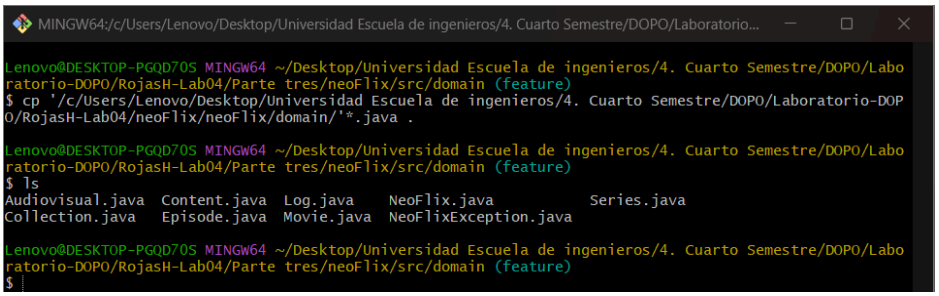
neoFlix

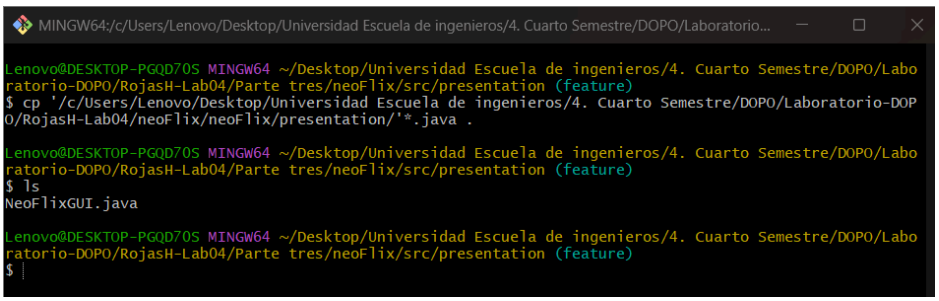
src

bin
doc
lib

1. Investiguen los archivos que deben quedar en cada una de esas carpetas y la organización interna de cada una de ellas.
    - src: Aquí van los archivos fuente, es decir, los .java escritos por el programador. Internamente se organiza en una subcarpeta por cada paquete. En neoFlix quedan src/domain y src/presentation porque cada archivo declara su paquete con la línea package y Java exige que la ruta de carpetas coincida con ese nombre.
    - bin: Aquí van los archivos compilados, es decir los .class (bytecode) que genera el comando javac. Internamente tiene la misma estructura de paquetes que src o sea bin/domain y bin/presentation. Así se mantienen separados los fuentes de los compilados.
    - doc: Aquí va la documentación del proyecto generada con el comando javadoc, en formato HTML a partir de los comentarios del código. Internamente tiene un archivo de entrada, index.html, y una carpeta por cada paquete con las páginas de cada clase. Esta estructura la crea javadoc automáticamente.
    - lib: Aquí van las librerías externas que necesita el proyecto, en archivos .jar. En neoFlix sería, por ejemplo, la librería de JUnit para las pruebas. Internamente no tiene subcarpetas, los .jar van sueltos.

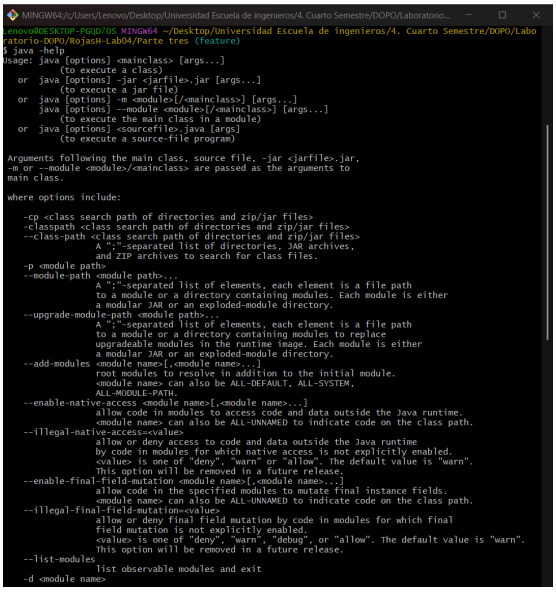
2. (a) ¿Qué archivos de neoFlix deberían estar en el directorio src? (b) ¿Cómo deberían estar organizados?
    a. Solo los `.java` de los paquetes `domain` y `presentation`
    b. Organizados por paquetes: `src/domain` y `src/presentation`
3. Copien a src únicamente los archivos de los paquetes de presentación y dominio. Consulten y capturen el contenido de los directorios que se modificaron.

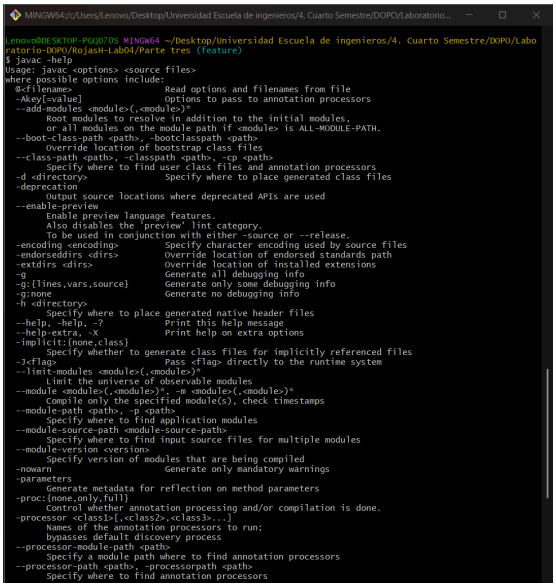




C Comandos de java [En lab04.pdf]

1. Consulten para qué sirven cada uno de los siguientes comandos:
   javac
   java
   javadoc
   jar
   - javac: es el compilador de Java. Toma los archivos fuente .java y los convierte en archivos .class (bytecode), que es lo que la máquina virtual puede ejecutar.
   - java: es el que ejecuta el programa. Lanza la máquina virtual de Java (JVM), carga las clases compiladas y arranca el programa desde la clase que tiene el método main.
   - javadoc: es el generador de documentación. Lee los comentarios Javadoc del código fuente y produce la documentación de la API en páginas HTML.
   - jar: es el empaquetador. Reúne los archivos .class (y otros recursos) en un solo archivo .jar, que sirve para distribuir y ejecutar el programa.
2. Creen una sesión de consola y consulten en línea las opciones de los comandos java y javac. Capturen las pantallas.





3. Busquen la opción que sirve para conocer la versión a que corresponden estos dos comandos. documenten el resultado.

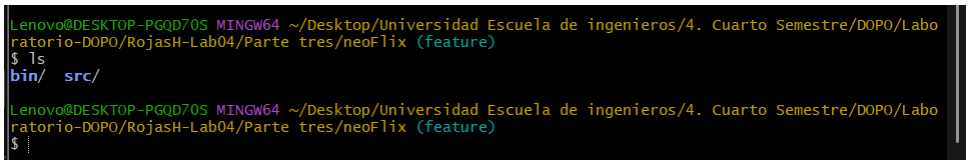
```
Lenovo@DESKTOP-PGQD7OS MINGW64 ~/Desktop/Universidad Escuela de ingenieros/4. Cuarto Semestre/DOPO/Laboratorio-DOPO/RojasH-Lab04/Parte tres (feature)
$ javac -version
javac 26.0.1

Lenovo@DESKTOP-PGQD7OS MINGW64 ~/Desktop/Universidad Escuela de ingenieros/4. Cuarto Semestre/DOPO/Laboratorio-DOPO/RojasH-Lab04/Parte tres (feature)
$ java -version
openjdk version "26.0.1" 2026-04-21
OpenJDK Runtime Environment (build 26.0.1+8-34)
OpenJDK 64-Bit Server VM (build 26.0.1+8-34, mixed mode, sharing)
```

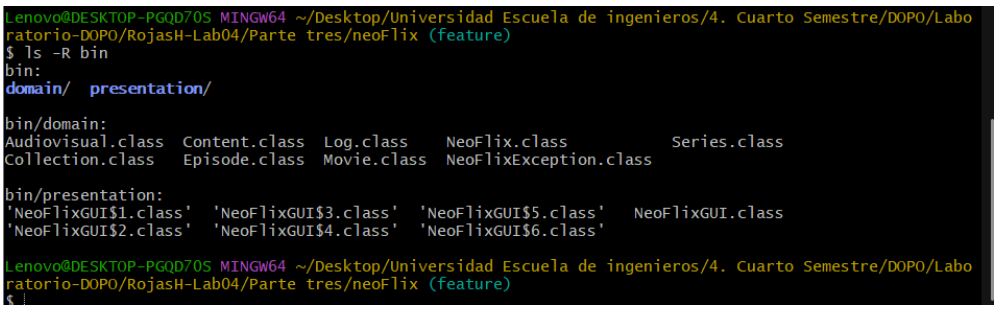
el compilador y la máquina virtual corresponden a la misma versión (26.0.1, OpenJDK), así que no habrá conflictos de versión al compilar y ejecutar.

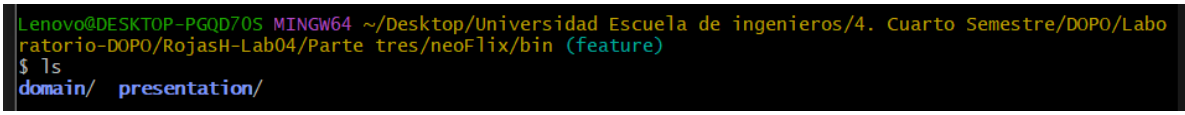
D Compilando [Enlab04.pdf]

1. Utilizando el comando javac, desde el directorio raiz (desde neoFlix con una sola instrucción), compile el proyecto. (a) ¿Qué instrucción completa tuvo que dar a la consola para compilar TODO el proyecto? Tenga presente que se pide un único comando y que los archivos compilados deben quedar en los directorios respectivos.
    a. javac -d bin src/domain/*.java src/presentation/*.java



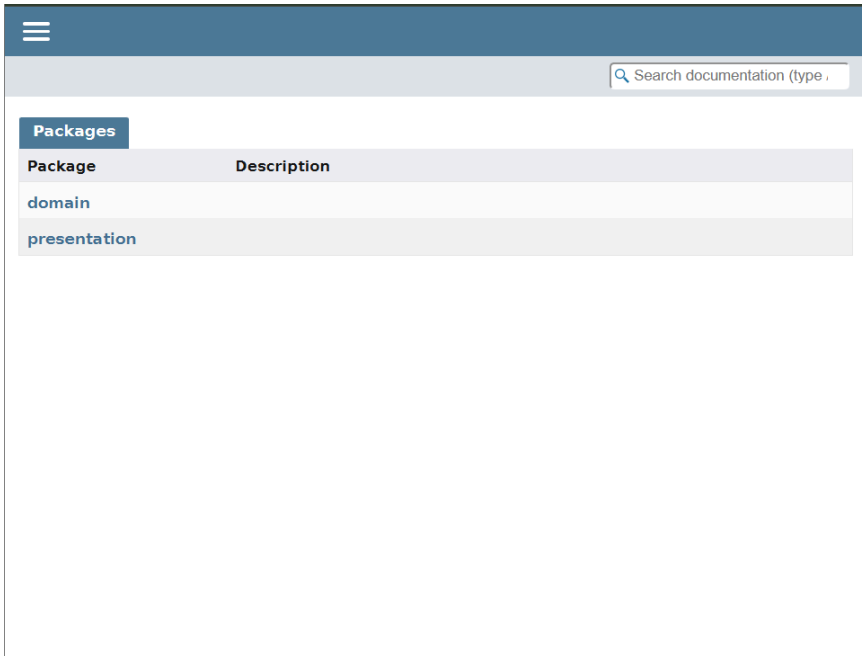
2. Revise de nuevo el contenido del directorio de trabajo y sus subdirectorios. (a) ¿Cuáles nuevos archivos aparecen ahora y dónde se ubican? Capturen las pantallas con los contenidos de los directorios de neoFlix/bin.
    a. Lo que se ve
        - bin/domain tiene 9 archivos .class, uno por cada clase del dominio: Audiovisual, Content, Log, NeoFlix, Series, Collection, Episode, Movie y NeoFlixException.
        - bin/presentation tiene NeoFlixGUI.class y seis archivos más: NeoFlixGUI$1.class hasta NeoFlixGUI$6.class.





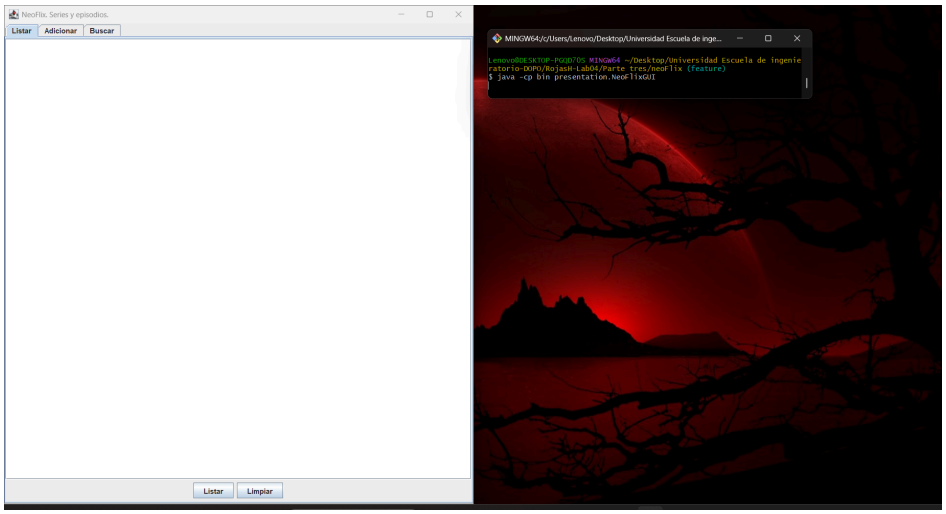
E Documentando [Enlab04.pdf]

1. Utilizando el comando javadoc, desde el directorio raiz, genere la documentación (API) en formato html, en este directorio. (a) ¿Cuál es el comando completo para generar esta documentación?
    a. javadoc -d doc -encoding UTF-8 src/domain/*.java src/presentation/*.java
2. (a) ¿Cuál archivo hay qué abrir para empezar a navegar por la documentación? Ábranlo y capturen la pantalla.
    a. El archivo de entrada es doc/index.html. Desde ahí se navega a los paquetes y a cada clase.



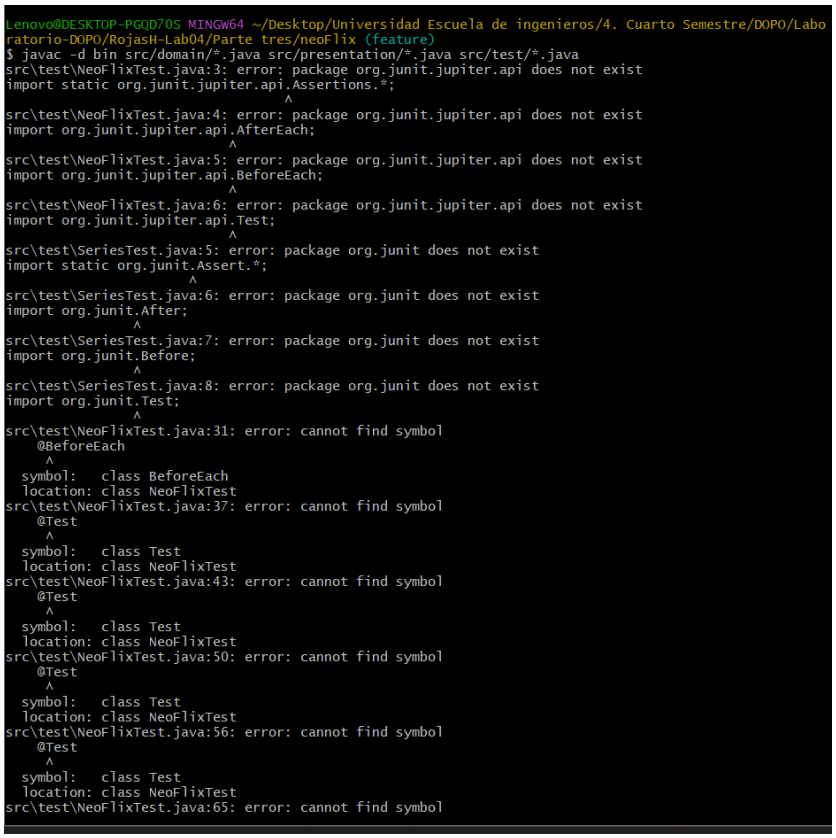
F Ejecutando [Enlab04.pdf]

1. Empleando el comando java, desde el directorio raíz, ejecute el programa. (a) ¿Cómo utilizó este comando?
    a. java -cp bin presentation.NeoFlixGUI. Se de la siguiente manera
        - java lanza la máquina virtual y ejecuta el programa.
        - -cp bin (classpath) le dice dónde buscar los .class en este caso en la carpeta bin.
        - presentation.NeoFlixGUI es el nombre completo de la clase que tiene el main: el paquete, un punto y el nombre de la clase. Va sin .class y sin barras de carpeta
2. Capturen la pantalla del programa en ejecución.

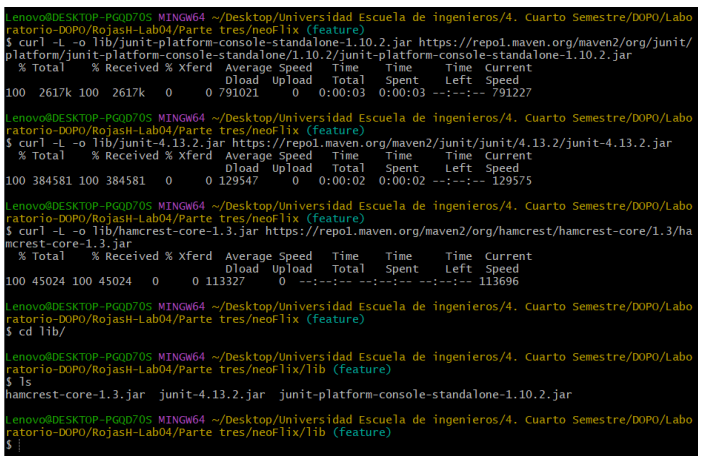


G Probando [Enlab04.pdf]

1. Adicione ahora los archivos del paquete de pruebas y trate de compilar nuevamente el programa. (a) ¿Qué pasa?
    a. javac -d bin src/domain/*.java src/presentation/*.java src/test/*.java
    Falla con errores del tipo package org.junit.jupiter.api does not exist. Las pruebas usan JUnit y el compilador no sabe dónde está esa librería.



2. Tenga en cuenta que estas clases requieren la librería junit. (a) ¿Dónde se incluye una librería en la estructura de archivos? (b) ¿Cómo se compila incluyendo un .jar? (c) ¿Qué instrucción completa tuvo que dar a la consola para compilar?
    a. Las librerías van en la carpeta lib.
    b. Para compilar con un .jar se usa la opción -cp (classpath), que le indica al compilador dónde buscar clases externas.
    c. javac -d bin -cp "lib/*" src/domain/*.java src/presentation/*.java src/test/*.java



3. Ejecute desde consola las pruebas. (a) ¿Cómo utilizó el comando java?.
    a. java -jar lib/junit-platform-console-standalone-1.10.2.jar -cp "bin;lib/junit-4.13.2.jar;lib/hamcrest-core-1.3.jar" --scan-classpath bin
    Explicación de que hace:
        - java -jar lib/junit-platform-console-standalone-1.10.2.jar lanza el ejecutor de pruebas de JUnit.
        - -cp "bin;lib/junit-4.13.2.jar;lib/hamcrest-core-1.3.jar" le dice dónde están tus clases compiladas y las librerías que necesitan las pruebas de JUnit 4. Se separan con ; porque estás en Windows, y van entre comillas para que Git Bash no se confunda.
        - --scan-classpath bin busca y corre todas las pruebas que haya en bin.
4. Pegue en su documento el resultado de las pruebas

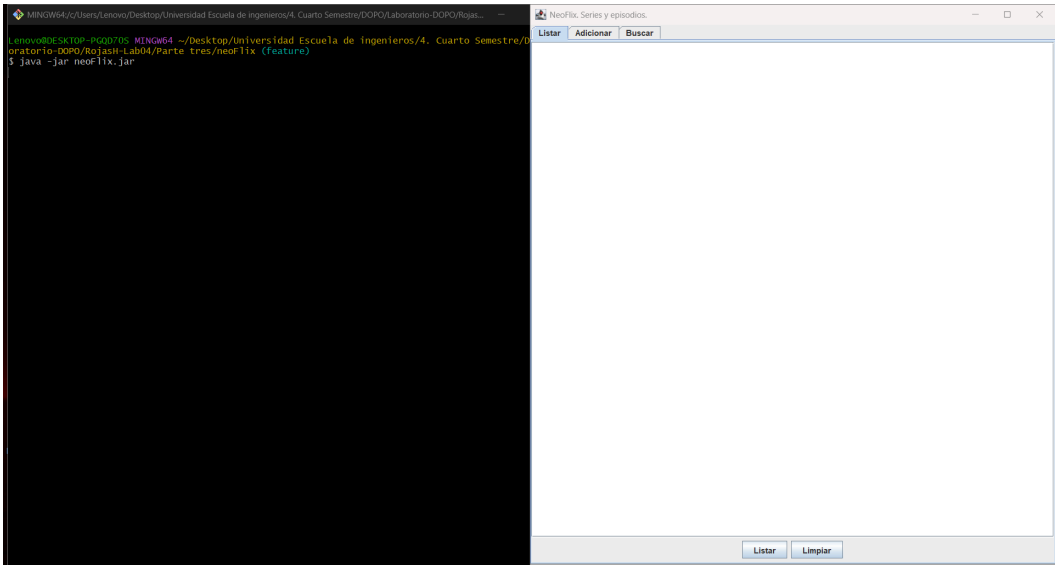


H Empaquetando [En lab04.pdf]

1. Consulten como utilizar desde consola el comando jar para empaquetar su programa entregable en un archivo .jar, que contenga los archivos bytecode necesarios (no las fuentes ni las clases de prueba)(a) ¿Cómo empaquetó jar ?
    a. jar cfe neoFlix.jar presentation.NeoFlixGUI -C bin domain -C bin presentation

```
Lenovo@DESKTOP-PGQD70S MINGW64 ~/Desktop/Universidad Escuela de ingenieros/4. Cuarto Semestre/DOPO/Laboratorio-DOPO/RojasH-Lab04/Parte tres/neoFlix (feature)
$ jar tf neoFlix.jar
META-INF/
META-INF/MANIFEST.MF
domain/
domain/Audiovisual.class
domain/Collection.class
domain/Content.class
domain/Episode.class
domain/Log.class
domain/Movie.class
domain/NeoFlix.class
domain/NeoFlixException.class
domain/Series.class
presentation/
presentation/NeoFlixGUI$1.class
presentation/NeoFlixGUI$2.class
presentation/NeoFlixGUI$3.class
presentation/NeoFlixGUI$4.class
presentation/NeoFlixGUI$5.class
presentation/NeoFlixGUI$6.class
presentation/NeoFlixGUI.class
```

2. (a) ¿Cómo se ejecuta el proyecto empaquetado? Capturen la pantalla del programa en ejecución.
    a. Se ejecuto con: java -jar neoFlix.jar



RETROSPECTIVA

1. ¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes? (Horas)
   El tiempo total fue de 19 horas
2. ¿Cuál es el estado actual del laboratorio? ¿Por qué?
   El laboratorio está terminado en sus tres partes. En la Parte I se compiló el proyecto con la clase NeoFlixException y se implementaron los métodos de Series. En la Parte II se hizo ingeniería inversa de adicionar, listar y consultar, se fortaleció la adición con excepciones y se registraron los errores inesperados en NeoSeries.log. En la Parte III se compiló, documentó, ejecutó, probó y empaquetó el proyecto desde la consola. Las pruebas de unidad pasan completas
3. Considerando las prácticas XP del laboratorio, ¿cuál fue la más útil? ¿por qué?

La práctica más útil fue las pruebas unitarias, que consisten en comprobar individualmente que cada parte del programa funciona correctamente

4. ¿Cuál consideran fue el mayor logro? ¿Por qué?
   Haber hecho robusta la funcionalidad de adicionar. Pasamos de un programa que guardaba datos incorrectos en silencio (nombres repetidos, valores no numéricos, valores incoherentes, episodios inexistentes) a uno que los rechaza con un mensaje claro al usuario.
5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?
   El mayor problema fue pasar de BlueJ a la consola, sobre todo al compilar y ejecutar las pruebas. Se resolvió descargando los tres .jar necesarios en la carpeta lib, compilando con -cp "lib/*" y ejecutando con el ejecutor de JUnit indicando las librerías en el classpath
6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?
   Hice bien la revisión de mi código y dividí mi tiempo. Me comprometo a no dejar los diagramas para lo último
7. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.
   Anthropic. (2026). Claude [Modelo de lenguaje]. https://claude.ai